

Giới hạn & phân tích rủi ro bảo mật

1. Giới hạn hiện tại

- **Không giới hạn số lượng tài sản** — `getAllProperties()` lặp qua toàn bộ `propertyCount` ; nếu số tài sản rất lớn, lời gọi `view` này có thể chậm/tốn gas đọc ở phía client (không ảnh hưởng đến an toàn tiền, chỉ ảnh hưởng hiệu năng đọc).
- **Chỉ hỗ trợ thanh toán bằng ETH** — không hỗ trợ stablecoin/ERC-20, không có oracle quy đổi giá.
- **Trọng tài do admin tự chỉ định** (`grantRole(ARBITER_ROLE, ...)`), không có cơ chế bầu chọn/luân phiên trọng tài phi tập trung — xem mục 3.

2. Rủi ro bảo mật đã xử lý

Rủi ro	Biện pháp đã áp dụng
Reentrancy khi chuyển ETH (<code>payRent</code> , <code>acceptSettlement</code> , <code>voteOnDispute</code> , <code>rentProperty</code>)	Modifier <code>nonReentrant</code> (OpenZeppelin <code>ReentrancyGuard</code>) + pattern checks-effects-interactions: cập nhật state trước, chuyển tiền sau. Xem chi tiết tại thiet-ke-du-lieu.md .
Chủ nhà tự thuê nhà của chính mình để thao túng trạng thái	<code>require(msg.sender != p.landlord)</code> trong <code>rentProperty</code> .
Người lạ trả tiền/xác nhận bàn giao/tất toán thay người có quyền	Mỗi hàm đều <code>require(msg.sender == ...)</code> đúng vai trò (<code>tenant</code> , <code>landlord</code> , hoặc <code>ARBITER_ROLE</code>).
Khấu trừ vượt quá tiền cọc đang giữ	<code>require(deductAmount <= p.depositHeld)</code> kiểm tra ở cả <code>proposeSettlement</code> lẫn <code>voteOnDispute</code> .
Chuyển tiền thất bại âm thầm	Kiểm tra giá trị trả về của <code>.call{value: ...}("")</code> bằng <code>require(ok, ...)</code> thay vì dùng <code>transfer()</code> / <code>send()</code> (tránh giới hạn 2300 gas cứng, đồng thời không bỏ sót lỗi).
Chủ nhà tự quyết định khấu trừ cọc mà người thuê không đồng ý — (trước đây liệt kê ở mục "rủi ro chưa xử lý", nay đã có giải pháp)	Xem mục 3.1 — cơ chế đề xuất/đồng ý/khiếu nại + trọng tài multisig.
Trọng tài đơn lẻ tự ý quyết định tranh chấp	Yêu cầu <code>arbiterApprovalsRequired</code> (mặc định 2) trọng tài độc lập cùng đồng thuận đúng 1 mức khấu trừ mới thực thi — 1 trọng tài không đủ quyền tự quyết. Trọng tài không được vote 2 lần cho cùng 1 tranh chấp (<code>hasVotedOnDispute</code>).
Thanh toán trễ hạn không bị ràng buộc gì — (trước đây liệt kê ở mục "rủi ro chưa xử lý", nay đã có giải pháp)	Xem mục 3.2 — <code>payRent</code> bắt buộc cộng thêm phạt nếu quá <code>nextDueDate</code> .

3. Chức năng nâng cao đã triển khai

3.1. Cơ chế trọng tài khi có tranh chấp (multisig cho giải ngân tiền cọc)

Thay vì `endLease` cũ (chủ nhà quyết định và thực thi ngay lập tức), luồng tất toán giờ gồm nhiều bước, có đường thoát cho cả 2 phía:

1. `proposeSettlement(id, deductAmount)` — chủ nhà đề xuất mức khấu trừ, **chưa chuyển tiền**.
2. `acceptSettlement(id)` — người thuê đồng ý → tất toán ngay theo đúng mức đề xuất.
3. `disputeSettlement(id)` — người thuê không đồng ý → chuyển sang trạng thái `Disputed`, không bên nào đơn phương quyết định được nữa.
4. `voteOnDispute(id, deductAmount)` — chỉ địa chỉ có `ARBITER_ROLE` gọi được. Mỗi trọng tài bỏ phiếu cho **1 mức khấu trừ cụ thể**; khi đủ số phiếu `arbiterApprovalsRequired` (đặt lúc deploy, mặc định 2) cùng đồng thuận **1 mức giống hệt nhau**, hợp đồng tự động tất toán theo mức đó. Đây chính là cơ chế "multisig cho giải ngân tiền cọc" theo yêu cầu — không cần `TimelockController` /contract multisig riêng, chỉ cần đếm phiếu on-chain.

`ARBITER_ROLE` cấp qua `AccessControl` (`DEFAULT_ADMIN_ROLE`, người deploy, tự `grantRole` thêm trọng tài khác). Đây là **1 trong 2 chỗ duy nhất dùng AccessControl** trong hệ thống (chỗ còn lại là `MINTER_ROLE` của `RentalAgreementToken`) — nghiệp vụ chính (đăng tin/thuê/trả tiền) vẫn hoàn toàn phi tập trung, không cấp phép.

3.2. Phạt thanh toán trễ

`Property.nextDueDate` ghi hạn trả tiền kỳ tiếp theo, cập nhật mỗi lần `rentProperty` (lần đầu) và `payRent` (các kỳ sau, `+= rentPeriod`). Nếu `payRent` được gọi sau `nextDueDate`, bắt buộc trả thêm `lateFeeBps` (đặt lúc deploy, mặc định 500 = 5%) trên `monthlyRent` — trả thiếu (không kèm phạt) sẽ bị `revert`, không có cách "lách" phạt.

4. Rủi ro còn tồn tại / chưa xử lý

- **Không có time lock cho quyết định trọng tài:** khi đủ phiếu, tất toán thực thi **ngay lập tức**, không có khoảng trễ để bên còn lại phản ứng nếu phát hiện trọng tài thông đồng — khác với thiết kế `TimelockController` từng đề xuất (mục cũ, nay đã chọn hướng đơn giản hơn: đếm phiếu trực tiếp thay vì hàng đợi có độ trễ).
- **Rủi ro tập trung quyền lực ở admin:** chỉ `DEFAULT_ADMIN_ROLE` (ví deploy) mới cấp được `ARBITER_ROLE` — nếu khoá admin bị lộ, kẻ tấn công có thể tự cấp quyền trọng tài cho mình. Xem thêm [production-checklist.md](#) (khuyến nghị chuyển admin sang ví multisig thật ở production).
- **Không có time lock / hạn chót cho `confirmHandover`:** người thuê có thể trì hoãn xác nhận bàn giao vô thời hạn, khiến chủ nhà không đề xuất tất toán được để cho thuê lại.
- **Xung đột lợi ích: contract không tự chặn chủ nhà/người thuê bỏ phiếu trọng tài cho chính tranh chấp của mình.** `voteOnDispute` chỉ kiểm tra `ARBITER_ROLE`, không kiểm tra `msg.sender != p.landlord && msg.sender != p.tenant`. Vấn đề này **đã xảy ra thật** trong lúc test: ví admin (đồng thời cũng là chủ nhà của nhiều tin đăng) tự bỏ phiếu được cho tranh chấp của chính mình, vì `ARBITER_ROLE` được cấp mặc định cho admin lúc deploy (`_grantRole(ARBITER_ROLE, admin)` trong constructor). Đã **giảm thiểu bằng quản trị vai trò** (không sửa contract, không cần deploy lại): thu hồi `ARBITER_ROLE` khỏi ví admin/chủ nhà (`scripts/revoke-arbiter.ts`), cấp lại cho một ví thứ 3 hoàn toàn độc lập, không phải chủ nhà/người thuê của bất kỳ tin nào. Đây chỉ là biện pháp **vận hành** (kỷ luật cấp quyền thủ công), không phải ràng buộc **cưỡng chế trên contract** — một admin tương lai vẫn có thể lờ cấp `ARBITER_ROLE` cho một địa chỉ đang là chủ nhà/người thuê. Cách khắc phục triệt để (chưa làm, cần sửa contract + deploy lại): thêm `require(msg.sender != p.landlord && msg.sender != p.tenant)` ngay trong `voteOnDispute`.

- **Front-running về lý thuyết:** hai người thuê cùng gửi `rentProperty` cho cùng một `id` gần như đồng thời — giao dịch tới sau sẽ tự động revert (vì `status` đã đổi thành `Active`), không mất tiền, nhưng trải nghiệm người dùng chưa tối ưu (không có hàng đợi/thông báo trước).
- **Dữ liệu `title / location / note` là `string` tự do:** không kiểm duyệt nội dung, có thể bị dùng để chèn nội dung không phù hợp (rủi ro về mặt vận hành, không phải bảo mật tiền).
- **Không giới hạn gas cho vòng lặp** trong `getAllProperties()` nếu tập dữ liệu lớn.
- **`rentPeriod / lateFeeBps / arbiterApprovalsRequired` cố định lúc deploy** (biến `immutable`) — không đổi được sau khi contract đã lên chain, kể cả bởi admin. Muốn đổi phải deploy lại contract mới.